

Le patron de conception Décorateur

Durée : 45 min

Crédit : <https://www.math.univ-paris13.fr/~chaussar/>

1 Instructions générales

1. Téléchargez l'archive zip associée à cet énoncé sur la plateforme pédagogique.
2. Ouvrez le projet dans l'IDE IntelliJ IDEA (ou autre).
3. Si vous ne l'avez pas encore fait, configurez votre IDE pour n'accepter que des attributs avec un identifiant commençant pas un '_'. (*Dans IntelliJ IDEA, menu Settings::Editor::Code Style::Java, ajoutez le caractère '_' dans field et static field. Puis, dans le menu Settings::Editor::inspection::Java::Naming Conventions ajoutez la vérification de Field naming convention (instance field et static field) avec l'expression régulière : `_[a-z][A-Za-z\d]*`.*)
4. Exécutez le programme pour vérifier que tout fonctionne correctement.

2 Données de l'exercice

Le code de l'archive fournie affiche l'image de René la Taupe grâce à la classe `ReneLaTaupe` et place un chapeau sur sa tête. Dans la suite du Coding Dojo, vous pouvez ignorer la classe `MyImage` qui affiche une image en utilisant la bibliothèque Swing.

3 Exercice 1

Modifiez le code du `Main` pour afficher, en plus du chapeau, des lunettes sur René sachant qu'elles doivent être positionnées aux coordonnées (255, 76). L'image de lunettes `sun_glasses.png` est dans le dossier `img`.

4 Exercice 2

On souhaite maintenant produire différents logos de René avec des accessoires. René pourra ainsi avoir des lunettes et un chapeau puis chanter, ou une sucrerie et un smiley.

Le dossier `img` contient les images correspondantes : la chanson se place à la position (10,

10), le bâton de sucrerie à la position (441, 202) et le smiley à la position (260, 210).

Écrivez un code permettant de générer facilement un René avec lunettes avec ou sans un chapeau ou avec ou sans une chanson... L'idée est de proposer aux développeurs une génération en une ligne d'un nouveau logo. On utilisera donc un décorateur sur la méthode `getLogo()`.

Pour cela, créez une classe abstraite commune entre tous les accessoires que René peut porter nommée `LogoAccessory`. Regroupez René et ses accessoires sous la même interface `SuperLogo`. Ensuite, regroupez les accessoires sous une classe abstraite `LogoAccessory`. Enfin, dites qu'`Accessoire` devra contenir un élément de type `SuperLogo`. Déclenchez des réactions en chaîne entre les `SuperLogo` pour superposer facilement tous les éléments à partir de l'appel de la méthode `getLogo()`.

5 Exercice 3

Afin de vérifier le respect du principe d'ouverture et de fermeture, implémentez aussi une fonction `getPrice()` qui permet de calculer le prix de tout un ensemble de logos (toujours à partir de réactions en chaîne). Chaque accessoire a un prix que vous inventerez et que l'on ajoute au prix de René pour former le prix total.

6 Exercice 4

Toujours pour vérifier le respect du principe d'ouverture et de fermeture, ajoutez le logo « Crazy Frog » et offrez à vos utilisateurs la possibilité de choisir entre des logos René la Taupe et des logos Crazy Frog. Pouvez-vous facilement ajouter une nouvelle image de base « Crazy Frog » à votre code ? (l'image `crazy_frog.jpg` se trouve dans le dossier `img`). Les accessoires se placent exactement à la même position pour René la Taupe et pour Crazy Frog.